

Integration & Release Sub-Team

...

Justin Ma • Dai Toan Dang • Anh Tuan Vo

March 3, 2026

Overview

Integration & Release

1. Our Mission & Responsibilities — Justin
2. How Code Gets to `Main` (branching, MRs, reviews) — Justin
3. Case Diagram for the Integration & Release Pipeline — Tuan
4. `README.md` — Dai

Other

5. Case Diagram for Identity Management — Tuan
6. Class Diagram for the Authentication Manager — Dai

Why Does I&R Exist:

Main always compiles.

Main always runs.

Main always passes.

Our Pillars of Responsibility

Merge Oversight

Review, verify, and execute
all merges into main

Owner:

All three of us

Release Readiness

Build instructions,
dry-runs, documentation

Owner:

Dai → Build & README

Tuan → Changelog &
documentation

Cross-Team Coordination

Scheduling merges,
resolving conflicts,
tracking issues

Owner:

Justin

The Feature-Branch Workflow

``main` (protected)` ← Merge Request ← ``feature branch` (short-lived)`

Branch Naming

`<team>/<type>/<issue#>-<short-description>`

Example:

``platform/feature/42-room-lifecycle``

Rules

- No direct pushes to main
 - Everything goes through an MR
- 5-day max lifespan per branch
- Sync with main at least every 3 days

Before You Open an MR

- ❑ Pull and merge latest `main` into your branch
- ❑ `./mvnw clean compile` → BUILD SUCCESS
- ❑ `./mvnw test` → all tests pass
- ❑ `./mvnw javafx:run` → app launches
- ❑ Manually verify your change works

If any step fails , fix it first, don't open the MR

Side note: Use the MR template. It's in the Integration & Release Plan and will be in GitLab

Who Reviews What

Standard Change

1 peer from same sub-team

Shared Interface
Change

1 peer + 1 from each affected team

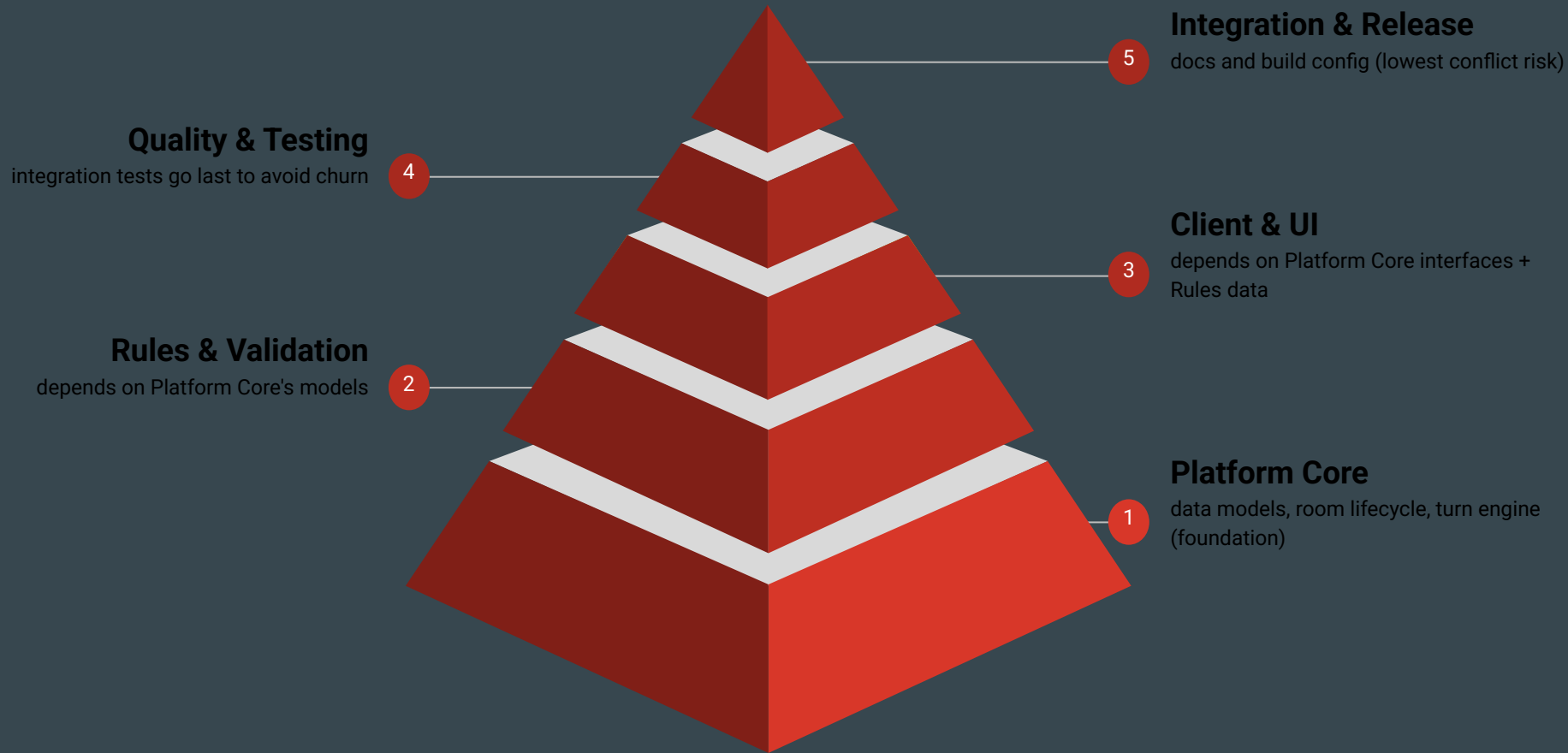
Documentation Only

1 peer from any team

I&R Changes (build,
README, etc.)

1 I&R member + 1 from any other team

Integration Day Merge Priority

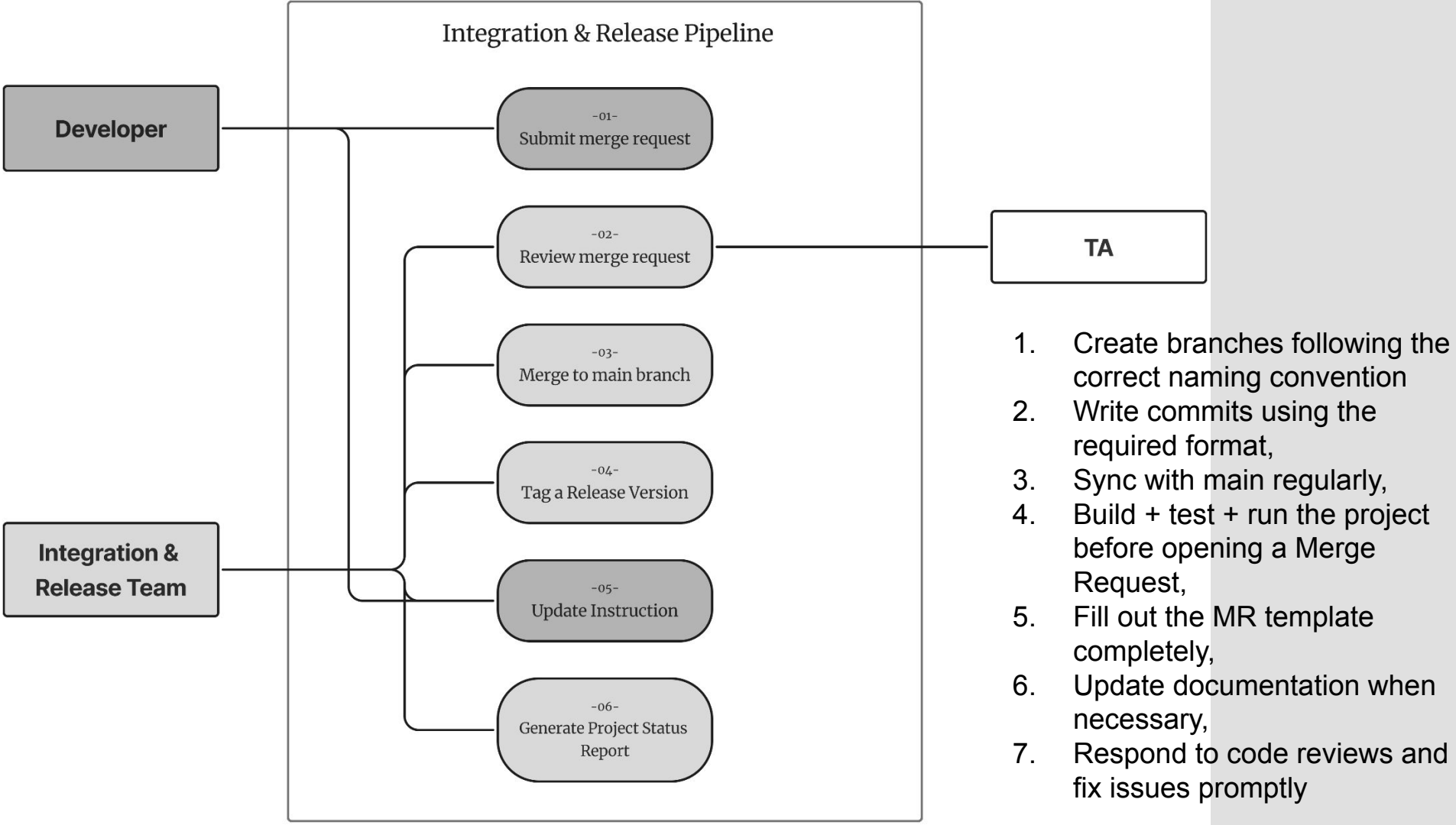


What We Need From Every Sub-Team

1. **Designate a Representative** : one person as your main point of contact with I&R
2. **Show Up to Integration Sessions** : bring your MRs ready to merge to main
3. **Follow the Standards** : branch naming, commit messages, MR template, pre-merge checklist

Use Case Diagram: Integration & Release Pipeline

Presenter: Anh Tuan Vo



README & Project Structure Overview

Presenter: Dai Toan Dang

Versioning

This project uses semantic versioning (vMAJOR.MINOR.PATCH). See the [Integration & Release Planning Document](#) for details.

For example: V1.2.30

Run Instructions

Option A - Run in IntelliJ (recommended)

1. Open the Maven tool window
2. Expand: Project -> Plugins -> javafx
3. Double-click: javafx:run

A JavaFX window should open.

Option B - Run from the Terminal (Maven Wrapper)

This project uses the Maven Wrapper. You do not need to install Maven.

Windows (PowerShell):

```
.\mvnw.cmd javafx:run
```

macOS or Linux:

```
./mvnw javafx:run
```



Current File ▾

[Start Free Trial](#)

2. Open in IntelliJ IDEA

- Select *File > Open* and navigate to the cloned folder.

3. Build the project

- Select *Build > Build Project*

Run Instructions

Option A - Run in IntelliJ (recommended)

1. Open the Maven tool window
2. Expand: Project -> Plugins -> javafx
3. Double click javafxrun



Current File ▾



Start Free Trial

Maven

▼ **m** javafx-starter

> Lifecycle

▼ Plugins

> clean (org.apache.maven.plugins:maven-clean-plugin:3.2.0)

> compiler (org.apache.maven.plugins:maven-compiler-plugin:3.13.0)

> deploy (org.apache.maven.plugins:maven-deploy-plugin:3.1.2)

> install (org.apache.maven.plugins:maven-install-plugin:3.1.2)

> jar (org.apache.maven.plugins:maven-jar-plugin:3.4.1)

▼ javafx (org.openjfx:javafx-maven-plugin:0.0.8)

m javafx:jlink**m** javafx:run

> resources (org.apache.maven.plugins:maven-resources-plugin:3.3.1)

Project Structure (Draft)

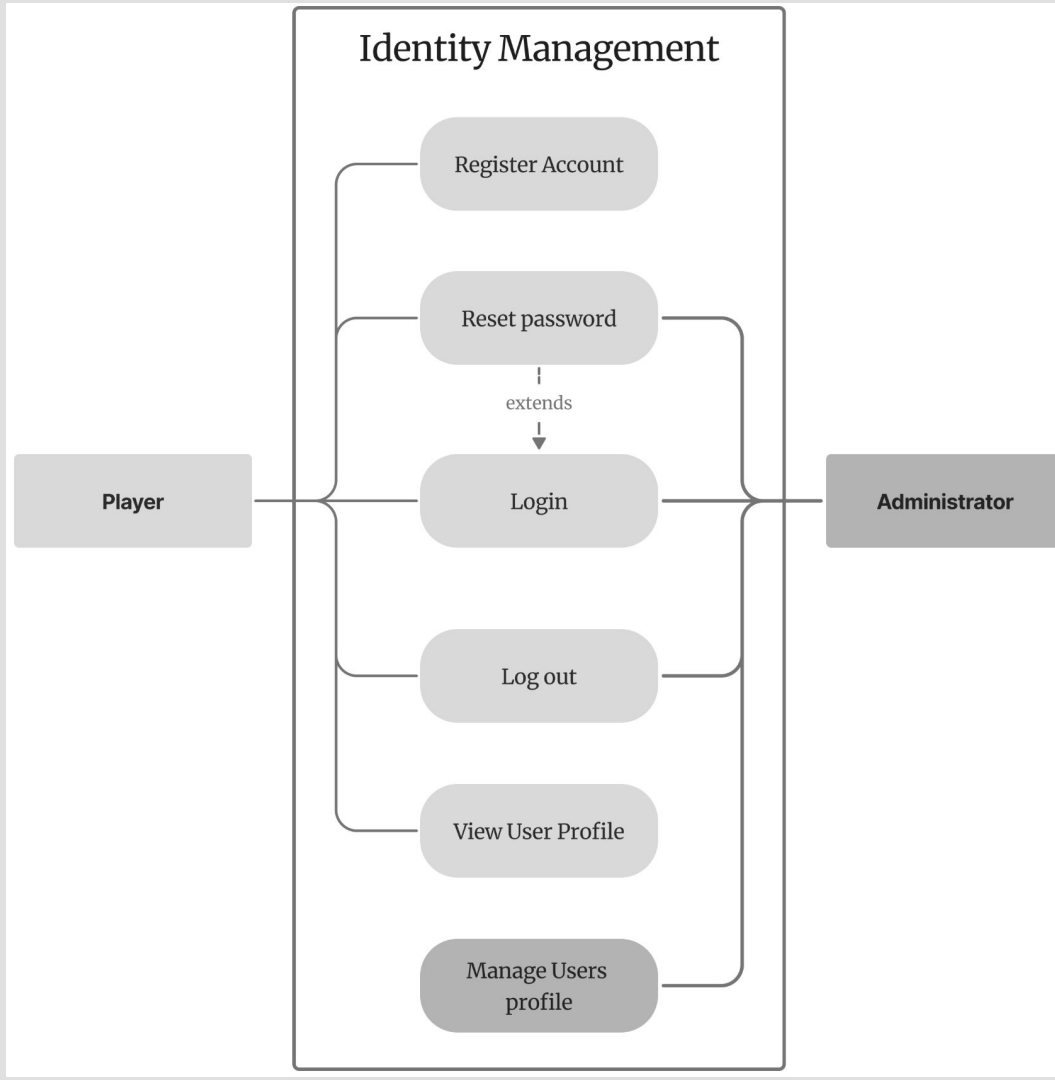


```
/
|__ src/           # Source code
|__ test/          # Test classes
|__ docs/          # Design and planning documents
|__ README.md
```

Brief descriptions of top-level directories will be updated as the project evolves.

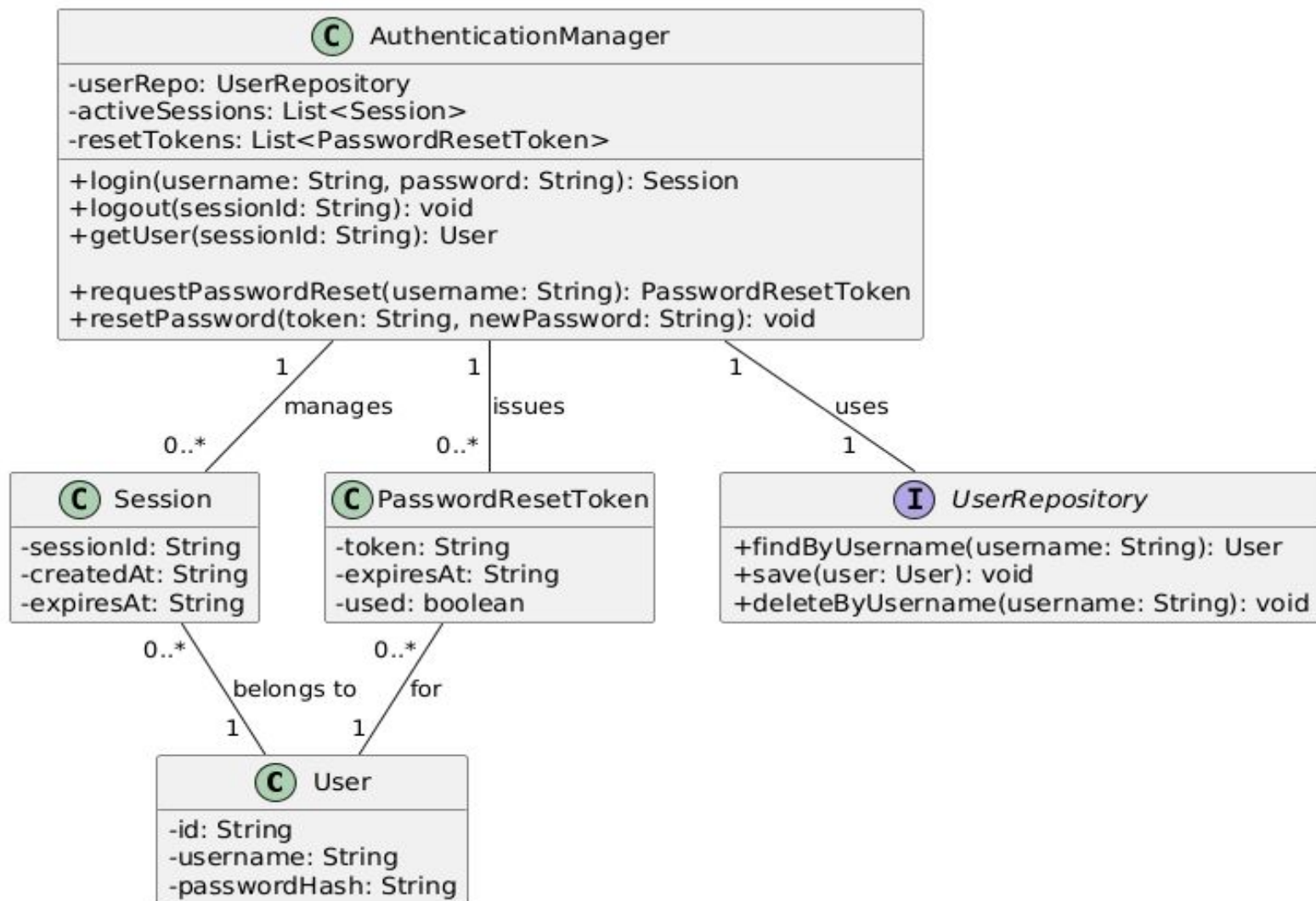
Use Case Diagram: Identity Management

Presenter: Anh Tuan Vo



Class Diagram: Authentication Manager

Presenter: Dai Toan Dang



Summary and Questions

Recap

1. Follow the workflow
2. Merge early, merge often
3. Designate your rep

Contact

justin.ma3@ucalgary.ca: cross-team coordination, merge oversight, interface reviews, integration issue tracking

daitoan.dang@ucalgary.ca: build and release, README maintenance, release dry-runs, build verification

anhtuan.vo@ucalgary.ca: documentation CHANGELOG, CURRENT_STATE updates, integration health reports